

한봉훈 · 백엔드 개발자

bng4535@gmail.com · 010-5751-4535 · [Blog](#) · [GitHub](#)

상품과 예약, 쿠폰처럼 변경이 여러 데이터에 걸쳐 전파되는 도메인을 다뤄온 3년 차 백엔드 개발자입니다. 데이터 모델과 DB 제약, 트랜잭션 경계로 변경 범위와 중복 처리를 제한하고, 외부 연동 실패와 장애 원인을 운영자가 추적할 수 있도록 상태와 알림을 남겨 왔습니다.

이 경험을 주문과 클레임 연계가 많은 위버스 커머스의 실패 처리와 운영 개선에 활용하고 싶습니다.

기술 스택

- **Backend:** Java · Spring Boot · JPA · QueryDSL · REST API
- **Data·Messaging:** PostgreSQL · MySQL · Redis · Kafka · AWS SQS
- **Infrastructure·Operations:** AWS ECS/RDS · Docker · Terraform · Jenkins · Airflow
- **Testing·Observability:** Testcontainers · WireMock · CloudWatch · Grafana · Loki · Tempo
- **AI Development:** Claude Code · Codex · 명세 기반 작업 분해와 검증

경력

미니슈테크놀로지 — Backend Developer · 2025.04 ~ 현재

프리미엄 안티에이징 예약 플랫폼 **클럽미니쉬**의 백엔드 개발과 운영을 맡고 있습니다.

Java 21 · Spring Boot 3.5 · JPA · QueryDSL · PostgreSQL · Redis · AWS ECS on EC2 · AWS RDS · AWS SQS · Terraform · Jenkins · Airflow

상품과 예약, 혜택 도메인

- **쿠폰 도메인 신규 설계:** 의료법 검토로 적용 대상 범위가 마지막까지 확정되지 않아, 대상 타입과 적용 조건을 분리해 범위를 데이터로 표현했습니다. 그 결과 배포 1주일 전 기획 변경을 코드 수정 없이 당일 반영했고 이후 변경도 같은 방식으로 처리했습니다. 공개 선착순이 아닌 초대 발급이라 동시 경쟁이 크지 않아, 발급 수량과 1인 1매를 조건부 UPDATE와 DB 제약으로 처리해 재고 초과나 중복 발급 없이 운영했습니다.
- **상품 데이터 모델 개편:** 템플릿은 기존 상품을 기반으로 빠르게 여러 개를 만들려고 도입됐는데, 참조 구조여서 한 상품의 가격을 고치면 같은 템플릿을 쓰는 다른 상품까지 바뀌었습니다. 일괄 수정 니즈가 없음을 확인하고 생성 편익의 복사 방식으로 남겼습니다. 템플릿 테이블을 상품 테이블로 병합하면서 운영 데이터를 이관하고 이미지 FK를 재지정한 뒤 관련 테이블을 정리해, 빠른 생성은 유지하면서 상품 간 수정 전파를 없앴습니다.

외부 시스템 연동과 장애 대응

- **분산된 예약 창구 단일화와 EMR 연동** (4개 지점, 약 7개월, 동기화 약 5,000건): 예약이 다섯 경로로 들어오는데 중앙 관리 팀과 병원 현장이 서로 다른 도구를 써서 통일할 수 없었습니다. 우리 시스템을 기준으로 양방향 동기화를 설계하고, 멍든기가 없어 재전송이 중복 예약을 만들 수 있었으므로 반영 여부를 확인할 수 없는 요청은 어드민에 노출해 운영자가 처리하도록 했습니다. 양쪽 도구를 유지하면서 예약 상태를 중앙에서 추적할 수 있게 했고, EMR 장애가 사용자 예약 접수를 막지 않게 했습니다.
- **5xx 진단과 이슈 생성 자동화**: 래핑된 예외가 SQL 메시지에 가려 근본 원인이 알림에 남지 않아, 매번 Grafana를 확인해야 했습니다. 예외 원인과 traceId를 함께 알리고 같은 오류의 반복은 해시로 억제했으며, traceId로 로그와 배포 커밋을 모아 LLM이 원인 후보와 확인할 지점을 정리해 Slack과 GitHub 이슈로 남기도록 했습니다. 알림 경로는 비동기로 격리해 Redis나 Slack 장애가 요청 처리로 번지지 않게 했습니다.

개발 환경과 AI 협업

- **테스트 환경 재구축**: AWS와 외부 DB에 의존해 로컬과 CI에서 결과가 달랐던 테스트를 Testcontainers 기반으로 바꿨습니다. 실패하던 162개를 복구해 전체 1,792개가 로컬과 CI에서 같은 결과로 통과하며, 같은 Job의 실행 시간은 2분58초에서 1분26초로 줄었습니다.
- **빌드와 관측 정리**: Jenkins Multibranch와 ECS 배포 파이프라인을 정리하고 Docker 레이어 캐싱과 이미지 경량화를 적용해 동일 노드의 빌드 기록을 8분40초에서 2분10초로 줄였습니다. 상시 올리던 알람은 3일간 측정 한 기준값으로 다시 잡고, RDS 관측 설정을 Terraform으로 관리합니다.
- **AI 컨텍스트 문서 구조 개편**: 모든 도메인 지식이 진입 문서에 쌓여 작업과 무관한 내용까지 매번 읽히고 규칙이 충돌했습니다. 공통 규칙만 진입 문서에 남기고 설계 결정은 각 도메인 코드 옆으로 옮겨, Claude Code와 Codex가 같은 기준에서 필요한 도메인 문서만 읽도록 바꿨습니다.

티맥스에이앤씨 — Backend Developer · 2023.05 ~ 2024.09

슈퍼루키상 · 2023년도 신입사원 최고 성과자

Java 17 · Spring Boot · JPA · QueryDSL · MyBatis · MySQL · Redis

- 여러 계열사 프로젝트가 사용하는 공통 라이브러리를 설계해 사내 저장소에 배포했습니다. 반복되던 파일 처리 절차를 템플릿 메서드와 제네릭으로 분리해 도메인별 구현량을 약 150줄에서 15줄로 줄였습니다.
- 사내 인증 흐름을 분석해 로컬 환경에서 Mock 인증 컨텍스트를 자동 주입하는 모듈을 만들고 테스트마다 인증 토큰을 하드코딩하고 삭제하던 작업을 없앴습니다.
- [mybatis-dynamic-sql #828](#) 문제를 리포팅하고 개선 방향을 제안했으며 해당 내용이 2.0.0 릴리스에 반영됐습니다.

사이드 프로젝트

Fanshop Backend

개인 프로젝트로 주문과 재고, 결제를 분리하고 Kafka 기반 Saga를 구성했습니다. 주문과 Outbox 저장을 같은 트랜잭션에 묶고 소비자는 복합 유니크 제약으로 중복 이벤트를 거부했습니다. 결제보다 재고를 먼저 확보하도록 순서를 바꾸고

결제 실패 시 예약 수량을 해제하는 흐름을 구현했습니다. 운영 배포 없이 결제를 시뮬레이션했으며 재고 동시 요청 100건은 테스트 환경의 검증 규모입니다. · [Repository](#)

Java 25 · Spring Boot 4.0 · Kafka · MySQL · OpenTelemetry

학력.자격

경희대학교 컴퓨터공학과 · 2017.03 ~ 2023.02

AWS Certified Solutions Architect – Associate · CKA · 정보처리기사 · SQLD · 삼성 SDS 알고리즘 특강 수료